MSC DATA SCIENCE - MSC FINAL PROJECT 2025-26

PROACTIVE FLAKY TEST PREDICTION

WITH EXPLAINABLE MACHINE LEARNING

MOHAMMAD KAMIL KARAYA | ID: 2369981 | MSC DATA SCIENCE (FT) - UNIVERSITY OF BIRMINGHAM DUBAI | SUPERVISOR: DR. ANIS ZARRAD

Why Flaky Tests Matter

Motivated by six years as a Senior Test Engineer at Resideo (a Honeywell company), where flaky tests were a daily, costly reality

1

What



A flaky test passes or fails inconsistently, with no code change between runs.

2

Cost



Teams re-run, quarantine, or delete flaky tests instead of fixing them - eroding trust in the whole test suite.

3

Goal



Predict which tests are likely flaky before they cause problems - and explain why, not just output a score.

3.7% of tests in the FlakeFlagger dataset are labeled flaky - a realistic, heavily imbalanced rate that reflects real-world engineering practice

Related Work: How This Compares

Published results on the same FlakeFlagger dataset - also why a direct F1 comparison needs care

Method	Year	F1 (cross-project)	Approach	Explainable?
FlakeFlagger [1]	2021	7%*	Random Forest, static + dynamic features	Partial
Flakify [2]	2022	73%	CodeBERT (LLM), source-code only, black-box	No
HgtFlaky [3]	2026	81%	Graph neural network, runtime evidence	No
DeepFlaky [4]	2026	93%	Hybrid deep representation learning	No
This project	2026	**15%**	Tuned RF, engineered features	Yes (SHAP)

Why the gap, and what this project adds

- Strict evaluation lowers the number: tested on all 23 projects, zero leakage — stricter than some published splits.
- High F1 can mislead: class imbalance can inflate F1 without real learning — confirmed here by near-random ROC-AUC on IDoFT (84.6% flaky).
- Beats the original baseline: 15% F1 vs. FlakeFlagger's own per-project F1 of 7%

FlakeFlagger F1 back-calculated from Flakify's reported delta (Fatima et al., 2022): F1=73% stated as +66pp above FlakeFlagger. This project = tuned RF + engineered features, after threshold tuning is 15%

Dataset: Why FlakeFlagger

Chosen for realistic imbalance, rich features, and comparability with published work

1

What



FlakeFlagger (Alshammari et al., 2021): 21,661 tests across 23 real open-source Java projects, labeled by rerunning each test 10,000 times.

2

Why this dataset



3.7% flaky reflects realistic imbalance. IDoFT, by contrast, is 84.6% flaky - too easy, and shown in this project to risk inflated results.

3

Why it fits this project



Rich static + dynamic features enable real feature engineering, and it's the most-cited benchmark - enabling direct comparison to Flakify, HgtFlaky, DeepFlaky.

10,000 reruns per test to establish reliable ground-truth flaky labels - not a one-off guess

Methodology

Dataset, feature engineering, and honest cross-project validation, end to end

01

Dataset



FlakeFlagger: 21,661 tests,
23 open-source Java projects,
3.7% flaky (realistic imbalance)

02

Feature Engineering



24 to 37 features: smell aggregation, coverage ratio, churn trends, log-transforms of skewed columns

03

Cross-Project Evaluation

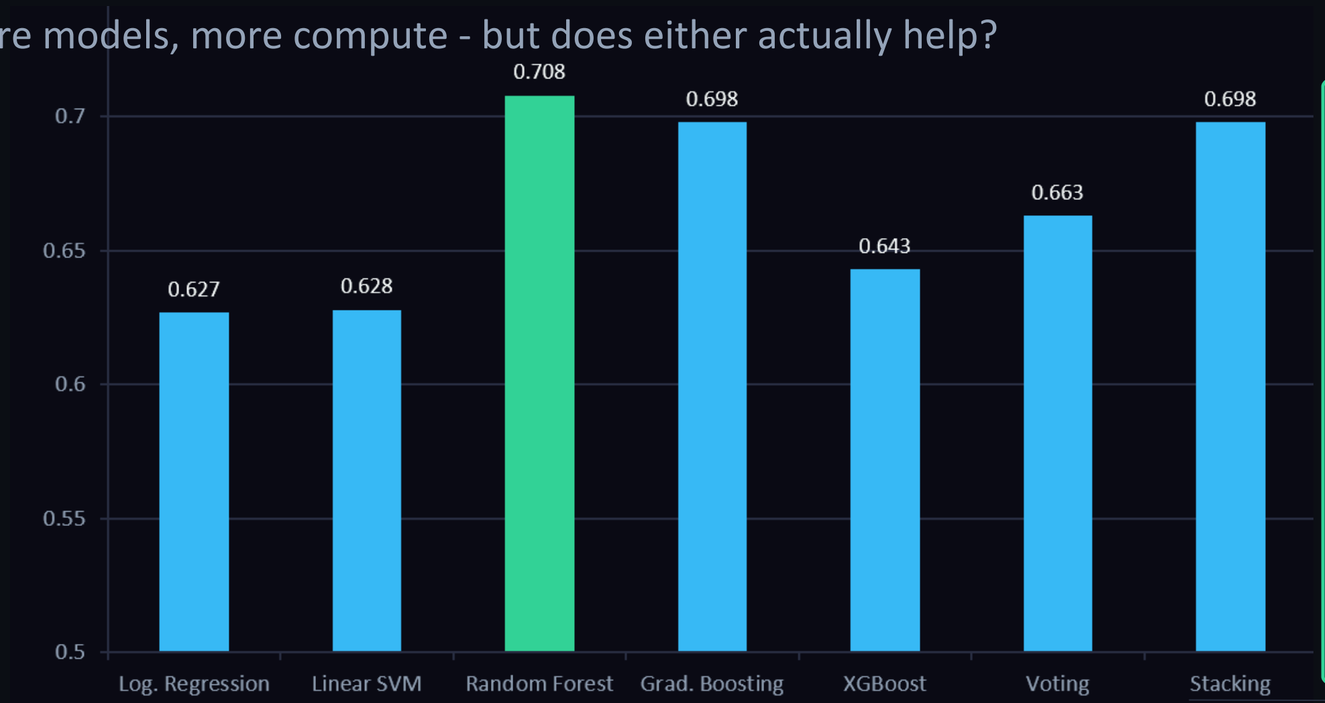


Leave-One-Group-Out by project - train on 22 projects, test on 1 unseen project, every fold

Why it matters: a random split would leak project identity into training and testing, inflating every score.

Model Comparison and Results

More models, more compute - but does either actually help?



Three findings worth highlighting

Random Forest leads every model tested: including a genuinely diverse ensemble (MLP, k-NN) added in a follow-up test.

Tuning had negligible effect: untuned and tuned Random Forest are statistically tied (0.702 vs 0.700 ROC-AUC).

Cost isn't justified by results: Stacking costs 5-8x more compute than Random Forest alone for a comparable ROC-AUC.

Configuration	ROC-AUC	PR-AUC	F1	Cost (23 folds)
Logistic Regression	0.627	0.162	0.134	9s
Random Forest (150 trees, untuned)	0.702	0.185	0.019	210s
Random Forest (tuned)	0.700	0.188	0.052	265s
Stacking (5 models)	0.701	0.195	0.160	1187s
Stacking (7 models)	0.698	0.191	0.163	1678s

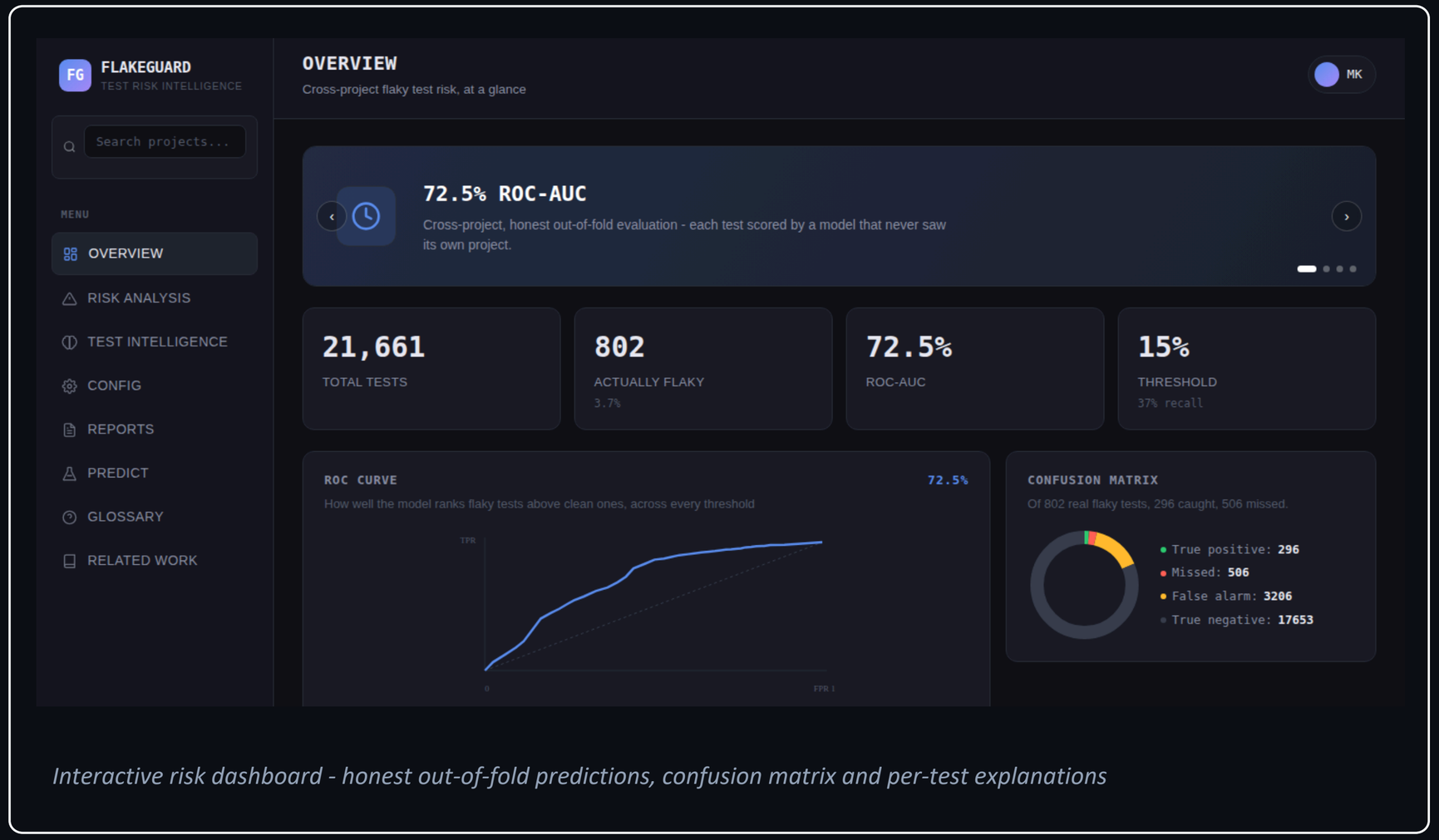
Explainability and Risk Dashboard

Not just a risk score - SHAP explains why and a live dashboard makes it explorable

What Drives Flaky Predictions (SHAP)

- **Execution timing**
  Strongest single signal
- **Code churn history**
  Recent modification activity
- **Coverage footprint**
  How much of the project a test touches
- **Test structure**
  Length, assertion density

Two of my own project-relative features (Execution_time_per_covered_line, execution_time_percentile_in_project) rank in the top 6 - the strongest validation yet of the v3 feature engineering.



Interactive risk dashboard - honest out-of-fold predictions, confusion matrix and per-test explanations

Conclusion and Future Work

Where this project landed, what it proves and what's left to do

Key Findings

- Random Forest (about 70% ROC-AUC) is close to the practical ceiling for this feature set - thoroughly tested against ensembling, including diverse model types
- Threshold tuning delivered a roughly 8x F1 improvement without changing the model at all*
- IDoFT tested as a secondary validation dataset (different imbalance, static-only features) - real but weaker signal, confirming FlakeFlagger as the right primary choice

~ 70%

ROC-AUC practical ceiling for this feature set

~ 8x

F1 improvement from threshold tuning alone, no model change

Next Steps

1. Extend SHAP and threshold tuning to IDoFT (secondary dataset)
2. Complete the written thesis

** Untuned Random Forest F1 = 0.019, tuned-threshold F1 = 0.154 (final_comparison.py / tune_threshold.py) - 0.154 / 0.019 ≈ 8.1x*

References

Published work cited in this presentation, in order of appearance

[1] Alshammari, A., Morris, C., Hilton, M., & Bell, J. (2021). *FlakeFlagger: Predicting Flakiness Without Rerunning Tests.* IEEE/ACM 43rd International Conference on Software Engineering (ICSE), 1572-1584. View →

[2] Fatima, S., Ghaleb, T. A., & Briand, L. (2022). *Flakify: A Black-Box, Language Model-based Predictor for Flaky Tests.* IEEE Transactions on Software Engineering. View →

[3] Dai, P., Ma, X., Zhao, Y., & Gong, Y. (2026). *Heterogeneous Graph Transformer with Multi-View Representation Learning for Flaky Test Detection* - Computers (MDPI Journals), 15(6), 372. View →

[4] Cai, J., Lei, Y., Li, Z., et al. (2026). *DeepFlaky: Deep hybrid representation learning for flaky test prediction.* Information and Software Technology, 193, 108070. View →



THANK YOU

MOHAMMAD KAMIL KARAYA | ID: 2369981 | MSC DATA SCIENCE (FT) - UNIVERSITY OF BIRMINGHAM DUBAI | SUPERVISOR: DR. ANIS ZARRAD